Hadoop WordCount on Project Gutenberg Datasets

CS6220 HW1

September 16, 2025

Abstract

This report presents two components. First, a Hadoop MapReduce *WordCount* pipeline: installing Hadoop on macOS (Apple Silicon), constructing scalable text datasets from Project Gutenberg, running the official *wordcount* example on three dataset sizes (*Small*, *Medium*, *Large*), post-processing outputs, and extracting run metrics. Second, an exploration of the K-Means clustering problem using `scikit-learn`: scaling the solution to progressively larger datasets and analyzing the effect of varying K-Means hyperparameters.

Contents

1 Problem Choice

Option 1.1 **2**

2 WordCount with Hadoop MapReduce **2**

2.1 Environment and Installation . . . 2

2.2 Datasets from Project Gutenberg . . . 2

2.2.1 Source and Rationale . . . 2

2.2.2 Dataset Construction Pipeline . . . 2

2.3 Running WordCount . . . 4

2.3.1 Driver Script and Parameters . . . 4

2.3.2 Post-processing and Outputs . . . 6

2.4 Results . . . 7

2.4.1 Top–K Words by Frequency . . . 8

2.4.2 First Words in Lexicographic Order . . . 9

2.4.3 Results . . . 9

3 K-Means Clustering with MapReduce **11**

3.1 Implementation Setup . . . 11

3.2 Experimental Design . . . 11

3.3 Results . . . 11

3.4 Discussion . . . 15

4 References **16**

1 Problem Choice

Option 1.1

I had no prior Hadoop experience, so I selected the option focused on Hadoop/MapReduce: implement and scale **WordCount** across increasingly large texts.

2 WordCount with Hadoop MapReduce

2.1 Environment and Installation

Platform: macOS (M-series). Hadoop installed via Homebrew.

Install (Homebrew):

```
brew install hadoop
```

Local single-node setup:

```
# Allow SSH to localhost (required by Hadoop scripts)
sudo systemsetup -setremotelogin on

ssh localhost

# Start HDFS (namenode + datanode) and check JVM procs
start-dfs.sh
jps
```

Relevant documentation: MapReduce Tutorial (Hadoop 1.x) and HDFS Commands (3.3.6).

2.2 Datasets from Project Gutenberg

2.2.1 Source and Rationale

To run WordCount we need large volumes of plaintext. Project Gutenberg offers a broad corpus ($\sim$76,734 books). See offline catalogs. Many books are available as UTF-8 plain text.

In practice, a common pattern for plaintext files is:

```
https://www.gutenberg.org/files/${id}/${id}-0.txt
```

Additional variants exist; the retrieval script includes basic fallbacks.

2.2.2 Dataset Construction Pipeline

I wrote a Bash script to fetch and merge books into three consolidated plaintext datasets of increasing size:

- **Small:** *284 MB*
- **Medium:** *1.32 GB*
- **Large:** *3.83 GB*

Retrieval:

Listing 1: build_gutenberg_datasets.sh

```bash
#!/usr/bin/env bash
set -euo pipefail

# Directory containing extracted RDF files (defaults to ./cache/epub)
RDF_DIR="${1:-./cache/epub}"

# Step 1: Collect unique ebook IDs
echo "[*] Collecting ebook IDs from ${RDF_DIR} ..."
IDS_FILE=ebook_ids.txt
grep -hoR 'ebooks/[0-9]\+' "$RDF_DIR" | awk -F'/' '{print $2}' | sort -n |
    uniq > "$IDS_FILE"
echo "[*] Found $(wc -l < "$IDS_FILE") ebook IDs."

# Step 2: Downloader for a single ebook ID
download_one() {
  id="$1"
  outdir="$2"
  mkdir -p "$outdir"
  urls=(
    "https://www.gutenberg.org/files/${id}/${id}-0.txt"
    "https://www.gutenberg.org/files/${id}/${id}.txt"
    "https://www.gutenberg.org/ebooks/${id}.txt.utf-8"
  )
  for u in "${urls[@]}"; do
    if curl -fsL --max-time 30 "$u" -o "${outdir}/${id}.txt" ; then
      if [ "$(wc -c < "${outdir}/${id}.txt")" -ge 5120 ]; then
        echo "OK ${id} via $(basename "$u")"
        return 0
      fi
    fi
  done
  echo "SKIP ${id}" >&2
  rm -f "${outdir}/${id}.txt" 2>/dev/null || true
  return 1
}

export -f download_one
export LC_ALL=C LANG=C

# Step 3: Define dataset sizes
SMALL_N=600
MEDIUM_N=3000
LARGE_N=9000

SMALL_DIR=data_gutenberg_small
MEDIUM_DIR=data_gutenberg_medium
LARGE_DIR=data_gutenberg_large

head -n "$SMALL_N"  "$IDS_FILE" > ids_small.txt
head -n "$MEDIUM_N" "$IDS_FILE" > ids_medium.txt
head -n "$LARGE_N"  "$IDS_FILE" > ids_large.txt
```

```
# Step 4: Download datasets
echo "[*] Downloading SMALL set..."
cat ids_small.txt  | xargs -n1 -P8 -I{} bash -c 'download_one "{}" "'"
   $SMALL_DIR"'" || true'
echo "[*] Downloading MEDIUM set..."
cat ids_medium.txt | xargs -n1 -P8 -I{} bash -c 'download_one "{}" "'"
   $MEDIUM_DIR"'" || true'
echo "[*] Downloading LARGE set..."
cat ids_large.txt  | xargs -n1 -P8 -I{} bash -c 'download_one "{}" "'"
   $LARGE_DIR"'" || true'

# Step 5: Concatenate text files
cat ${SMALL_DIR}/*.txt  > gutenberg_small.txt
cat ${MEDIUM_DIR}/*.txt > gutenberg_medium.txt
cat ${LARGE_DIR}/*.txt  > gutenberg_large.txt

# Step 6: Upload to HDFS
hdfs dfs -mkdir -p /datasets/gutenberg
hdfs dfs -put -f gutenberg_small.txt  /datasets/gutenberg/
hdfs dfs -put -f gutenberg_medium.txt /datasets/gutenberg/
hdfs dfs -put -f gutenberg_large.txt  /datasets/gutenberg/

echo "[*] Done. HDFS paths:"
echo "    /datasets/gutenberg/gutenberg_small.txt"
echo "    /datasets/gutenberg/gutenberg_medium.txt"
echo "    /datasets/gutenberg/gutenberg_large.txt"
```

Reference: Project Gutenberg Catalog Metadata. HDFS CLI reference: HDFS Commands.

2.3 Running WordCount

2.3.1 Driver Script and Parameters

I used the official Hadoop MapReduce *examples* JAR containing the `wordcount` demo. A helper script `wc_run.sh` orchestrates the run, cleans results, and collects metrics.

Running WordCount:

Listing 2: wc_run.sh

```
#!/usr/bin/env bash
set -euo pipefail

# Usage: ./wc_run.sh <INPUT> [REDUCERS=2] [TOP=30] [OUTDIR=./out]
INPUT="${1:-/datasets/gutenberg/gutenberg_small.txt}"
REDUCERS="${2:-2}"
TOP="${3:-30}"
OUTDIR="${4:-./out}"

JAR=/opt/homebrew/Cellar/hadoop/3.4.1/libexec/share/hadoop/mapreduce/
   hadoop-mapreduce-examples-3.4.1.jar
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
METRICS_FILE="$SCRIPT_DIR/wc_metrics.tsv"
LOG="$(mktemp)"
AGG="$(mktemp)"
```

```
HOUT="/tmp/$USER/wc_$(date +%s)"

# ----- name + output files -----
case "$INPUT" in
  *","*|*"*"*) NAME="combined" ;;
  *)           NAME="$(basename "$INPUT")" ;;
esac
mkdir -p "$OUTDIR"
LEX="$OUTDIR/${NAME}.lex.tsv"
BYC="$OUTDIR/${NAME}.bycount.tsv"

# ----- run WordCount (HDFS temp, no compression) -----
hdfs dfs -rm -r -f "$HOUT" >/dev/null 2>&1 || true
t0=$(date +%s)
set +e
hadoop jar "$JAR" wordcount \
  -D mapreduce.job.reduces="$REDUCERS" \
  -D mapreduce.map.output.compress=false \
  -D mapreduce.output.fileoutputformat.compress=false \
  -D mapreduce.input.fileinputformat.split.maxsize=$((64*1024*1024))   \
  -D mapreduce.input.fileinputformat.split.minsize=$((64*1024*1024))    \
  "$INPUT" "$HOUT" 2>&1 | tee "$LOG"
rc=${PIPESTATUS[0]}
set -e
[ "$rc" -eq 0 ] || { echo "job failed ($rc). see: $LOG"; exit "$rc"; }
t1=$(date +%s)
JOB_SECS=$((t1 - t0))

# ----- clean + aggregate (letters only, lowercase) -----
hdfs dfs -cat "$HOUT/part-r-"* \
| awk 'BEGIN{FS="\t"} $1~/^[A-Za-z]+$/ && $2~/^[0-9]+$/ {print tolower($1)
    ,$2}' \
| awk '{c[$1]+=$2} END{for(w in c) print w "\t" c[w]}' \
| sort -k1,1 > "$AGG"

cp "$AGG" "$LEX"
sort -k2,2nr "$AGG" > "$BYC"

# ----- counters we keep -----
RAW_UNIQ=$(awk -F'[= ]+' '/Reduce input groups=/{print $NF; exit}' "$LOG"
    || echo NA)
LAUNCHED_MAPS=$(awk -F'[= ]+' '/Launched map tasks[[:space:]]*=/{print $NF
    }' "$LOG" | tail -1 || true)
if [ -z "${LAUNCHED_MAPS:-}" ]; then
  LAUNCHED_MAPS=$(awk -F'[= ]+' '/Shuffled[[:space:]]+Maps[[:space:]]*=/{
      print $NF}' "$LOG" | tail -1 || true)
fi
: "${LAUNCHED_MAPS:=NA}"

CLEAN_UNIQ=$(wc -l < "$AGG")
CLEAN_TOTAL=$(awk '{s+=$2} END{print s+0}' "$AGG")

# ----- summary -----
echo "reducers: $REDUCERS"
```

```
echo "maps: $LAUNCHED_MAPS"
echo "job_secs: $JOB_SECS"
echo "raw_unique: $RAW_UNIQ"
echo "clean_unique: $CLEAN_UNIQ"
echo "clean_total: $CLEAN_TOTAL"
echo "saved: $LEX"
echo "saved: $BYC"

# top N (guard SIGPIPE)
set +e
head -"$TOP" "$BYC" || true
set -e

# ----- metrics TSV (next to script) -----
TS=$(date '+%Y-%m-%d %H:%M:%S')
if [ ! -f "$METRICS_FILE" ]; then
  printf 'timestamp\tdataset\treducers\tmaps\tjob_secs\traw_unique\
     tclean_unique\tclean_total\n' > "$METRICS_FILE"
fi
printf '%s\t%s\t%s\t%s\t%s\t%s\t%s\t%s\n' \
  "$TS" "$INPUT" "$REDUCERS" "$LAUNCHED_MAPS" "$JOB_SECS" \
  "$RAW_UNIQ" "$CLEAN_UNIQ" "$CLEAN_TOTAL" >> "$METRICS_FILE"
echo "metrics: $METRICS_FILE"

# ----- cleanup HDFS temp -----
hdfs dfs -rm -r -f "$HOUT" >/dev/null 2>&1 || true
```

Key parameters:

- **INPUT**: path to UTF-8 plaintext file.
- **REDUCERS**: number of reduce tasks (`mapreduce.job.reduces`).
- **TOP**: number of top words to keep in the frequency summary.
- **OUTDIR**: local directory to store processed outputs.
- Optional: control input split size, e.g.: `-D mapreduce.input.fileinputformat.split.maxsize=(64*1024*1024)` (64 MB for example).

2.3.2 Post-processing and Outputs

The script concatenates HDFS part files, then performs a simple cleanup to keep alphabetical tokens (lowercased) with numeric counts. It emits two TSV files per dataset:

- **Lexicographic:** `<name>.lex.tsv` (sorted A–Z by word).
- **By frequency:** `<name>.bycount.tsv` (sorted by descending count).

Metrics extracted from job logs:

- **RAW_UNIQ**: unique keys emitted before cleaning.
- **LAUNCHED_MAPS**: number of map tasks launched.
- **CLEAN_UNIQ**: unique words after cleaning.
- **CLEAN_TOTAL**: total token count after cleaning.
- We also get wall-clock time, input bytes, HDFS read/write, etc.

2.4 Results

We compare the three datasets (*small*, *medium*, *large*) under two criteria: (i) the *Top–K most frequent words*, and (ii) the *First words alphabetically*. Each table reports the entries side by side across datasets.

2.4.1 Top–K Words by Frequency

Table 1: Top 30 most frequent words in each dataset (from `*.bycount.tsv`).

Word	Small (count)	Medium (count)	Large (count)
the	2,634,701	12,567,835	36,130,419
of	1,411,092	7,030,922	19,937,932
and	1,509,838	6,609,810	18,785,080
to	1,141,379	5,525,038	15,966,696
a	936,888	4,444,499	12,106,816
in	740,194	3,532,980	10,152,734
that	497,281	2,317,132	6,612,406
he	516,309	2,260,590	6,434,770
i	515,815	2,299,516	6,163,988
was	485,220	2,117,445	6,080,344
his	399,485	1,884,797	5,414,452
with	372,837	1,711,169	4,877,677
it	355,160	1,581,160	4,605,778
for	341,382	1,556,603	4,487,785
as	313,707	1,516,017	4,330,169
is	283,495	1,446,499	4,186,503
had	301,044	1,308,849	3,852,723
you	273,012	1,272,160	3,642,482
not	243,962	1,184,322	3,463,476
at	245,944	1,154,891	3,321,964
be	235,025	1,099,673	3,232,992
but	232,436	1,073,637	3,079,714
on	226,866	1,076,427	3,061,130
by	204,132	1,059,838	3,018,881
her	235,691	971,362	2,969,342
she	235,380	896,069	2,687,217
this	181,769	905,578	2,656,999
have	190,824	908,850	2,639,735
which	174,393	893,758	2,600,439
or	176,262	1,016,312	2,531,022

2.4.2 First Words in Lexicographic Order

Table 2: First 25 words alphabetically in each dataset (from `*.lex.tsv`).

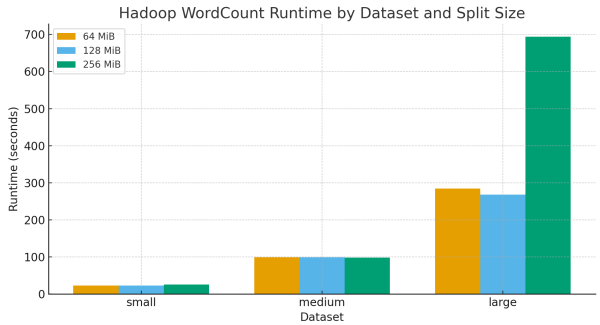
Word	Small (count)	Medium (count)	Large (count)
a	936,888	4,444,499	12,106,816
aa	85	151	256
aaa	1	1	7
aaanthor	14	14	14
aaas	6	6	6
aabborgt	1	1	1
aachen	4	14	85
aachens	1	1	1
aad	1	2	15
aage	1	3	4
aagesen	1	1	3
aahmes	1	4	7
aaiun	1	3	3
aak	1	1	3
aaland	1	3	3
aalborg	3	3	3
aalen	1	1	1
aalesund	3	3	3
aali	3	3	5
aamsi	3	3	3
aana	4	4	6
aap	8	8	44
aar	8	22	78
aarau	3	3	4
aarberg	1	1	1

2.4.3 Results

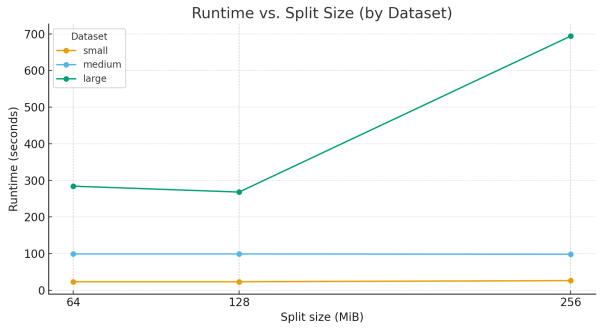
We evaluate *WordCount* on three Gutenberg subsets (*small*, *medium*, *large*) while varying the HDFS split size (64, 128, 256 MiB). Each larger dataset contains the smaller ones. As expected, the number of Map tasks follows the split size granularity: *small* has $(10, 6, 2)$ maps for $(64, 128, 256)$ MiB; *medium* $(40, 20, 10)$; and *large* $(116, 58, 30)$.

Runtime. For **small** and **medium**, runtime is essentially flat across split sizes (23–26 s and 98–99 s). Fixed overheads (job startup and shuffle) dominate, so additional parallelism brings little benefit. For **large**, split size matters a lot: **256 MiB** under-parallelizes (30 maps) and takes **694 s**, while **128 MiB** (58 maps) drops to **268 s** and **64 MiB** (116 maps) to **284 s**. The best trade-off is **128 MiB** with 64 MiB very close. See Figure 1a and Figure 1b.

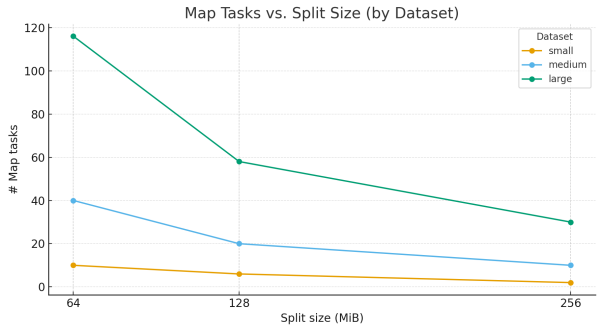
Scaling and parallelism. At a fixed split size (128 MiB), time grows near-linearly with data size: small→medium increases clean tokens by ×4.69 and time by ×4.30; medium→large is ×2.90 tokens for ×2.71 time. Figure 1c confirms that smaller splits yield many more map tasks, and Figure 1d



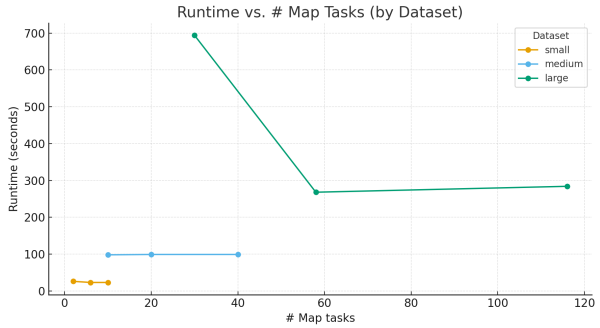
(a) Runtime by dataset and split size.



(b) Runtime vs. split size.



(c) Map tasks vs. split size.



(d) Runtime vs. map tasks.

Figure 1: Performance characteristics across datasets.

shows diminishing returns beyond a point (256 vs. 128 MiB on *large*): added coordination overhead offsets extra parallelism.

3 K-Means Clustering with MapReduce

3.1 Implementation Setup

For this experiment, I implemented K-Means clustering using Python's `scikit-learn`. Datasets were generated with the `make_blobs` utility to create synthetic points of different sizes and dimensionalities. Since this was done in Python, the notion of "JVM queue size" does not apply. To meet the scaling requirement, I progressively increased dataset size (5k, 50k, 500k, 2M samples) and dimensionality (4D vs. 10D).

Table 3: Synthetic datasets used for K-Means experiments (generated with `make_blobs`).

Name	**Samples**	**Dimensions**	**Clusters (centers)**
tiny (5k, 4D)	5,000	4	5
small (50k, 4D)	50,000	4	5
medium (500k, 4D)	500,000	4	5
large (2M, 4D)	2,000,000	4	5
medium, high dimensionality (500k, 10D)	500,000	10	5

Reference: KMeans, MiniBatchKMeans, make_blobs.

3.2 Experimental Design

I compared two implementations:

- Full `KMeans`,
- `MiniBatchKMeans`.

Each was run on all datasets with different numbers of clusters ($k \in \{3, 5, 7\}$). To study initialization strategies, I compared:

- `k-means++` (default, optimized seeding),
- `random` (random centroids),
- `custom` (first k samples as centroids, a naive baseline).

Execution time was measured with Python's `time` module, and clustering quality was evaluated using the inertia score (sum of squared distances to centroids). The code used to run experiments and plot results is included in the notebook deliverables.

3.3 Results

Scaling across dataset sizes. Figure 2 compares runtimes between standard K-Means and MiniBatch K-Means. MiniBatch K-Means scales much better on larger datasets (up to 2M samples), with runtime consistently one order of magnitude lower than full K-Means. The performance gap becomes even larger on high-dimensional datasets, where MiniBatch K-Means maintains a clear advantage over standard K-Means.

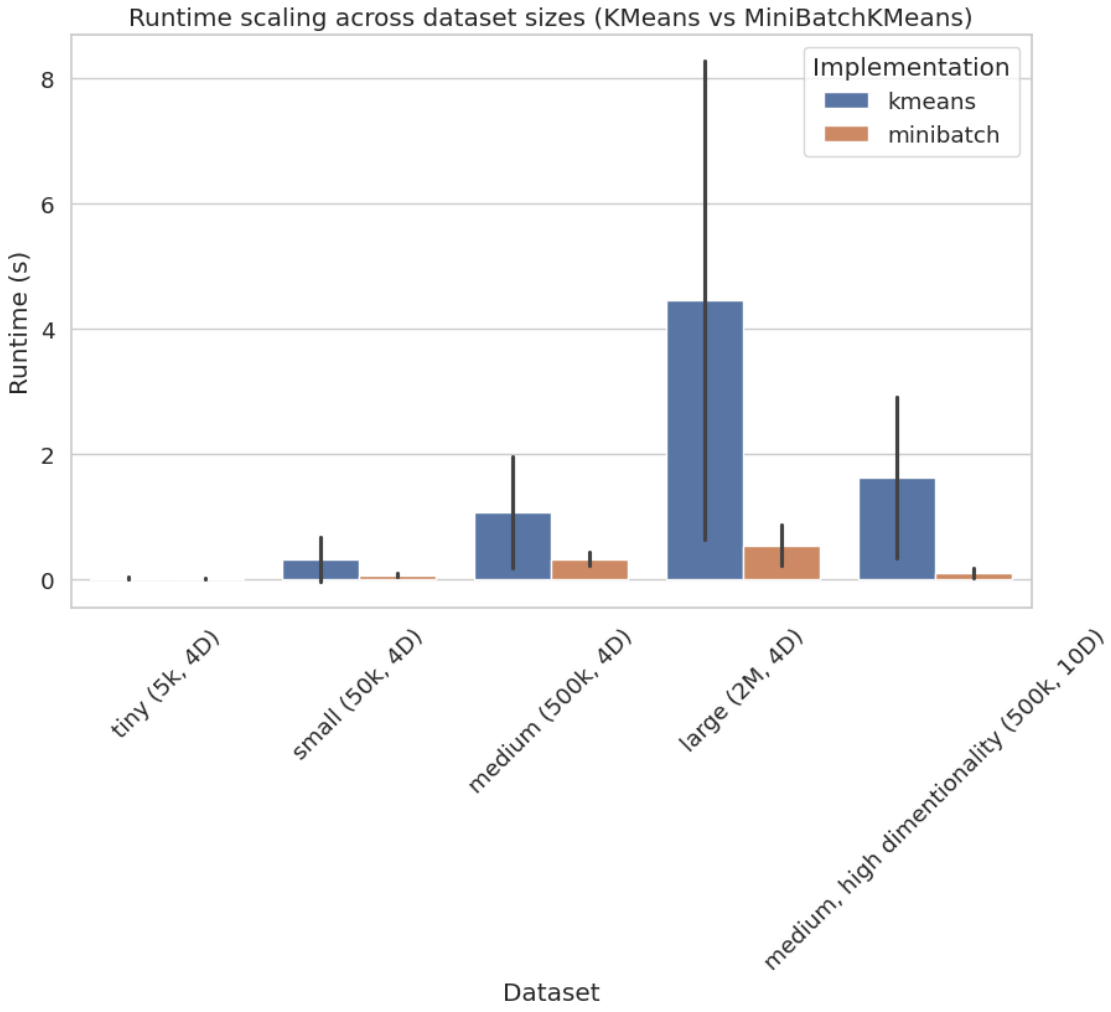


Figure 2: Runtime comparison across dataset sizes (independent scales).

Effect of initialization. Figure 3 shows the effect of different initialization methods on runtime. `k-means++` initialization tends to be slower but provides stable clustering quality. `random` is faster but less consistent, while `custom` is extremely fast mainly because, unlike the others, it does not repeat the initialization process multiple times (10) by default.

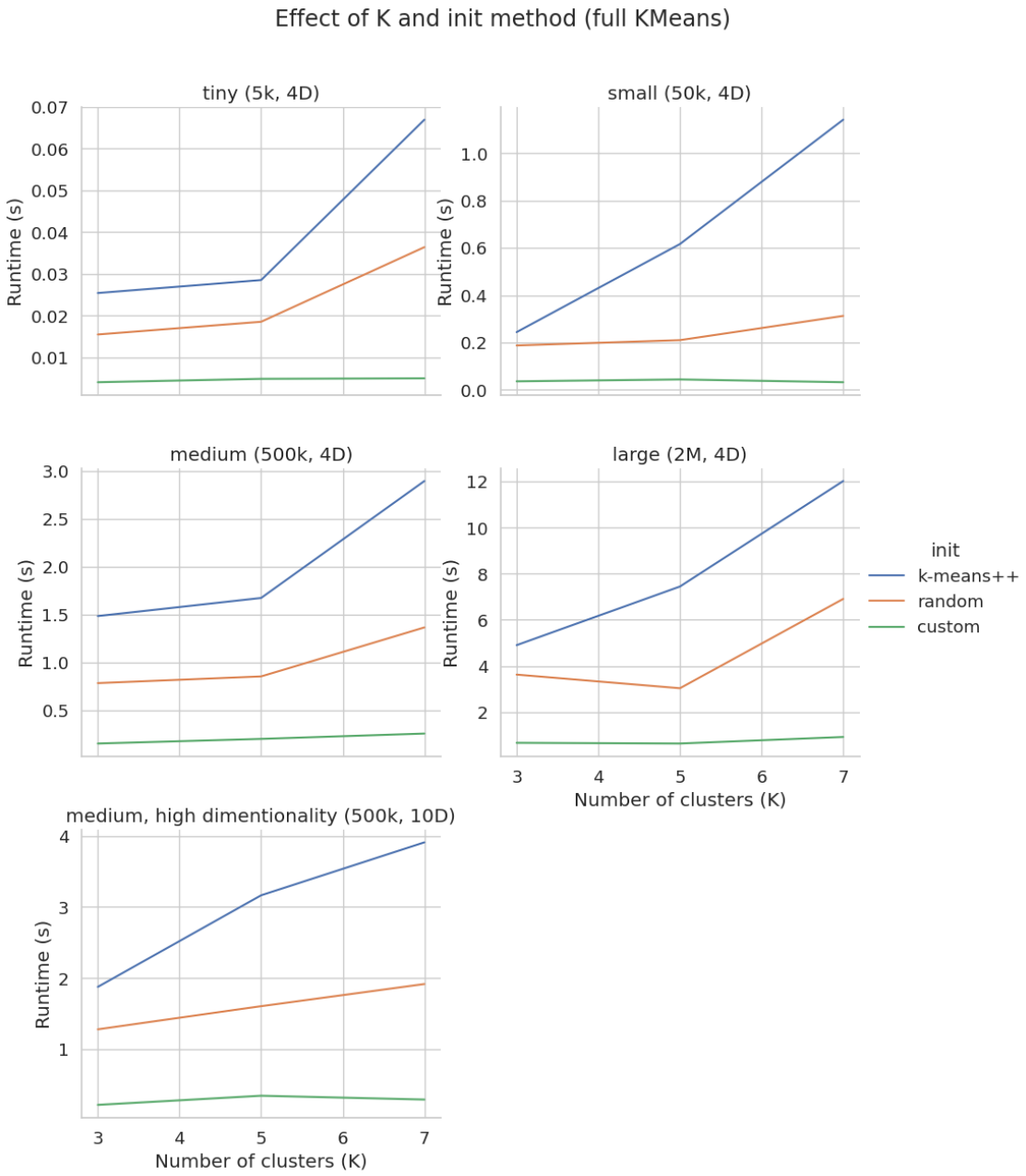


Figure 3: Effect of K and initialization method on K-Means runtime.

Runtime vs. number of clusters (K). To complement the scaling results, we also measured how runtime increases with the number of clusters. As expected, both K-Means and MiniBatch K-Means exhibit longer runtimes as K grows, since more centroid updates are required. However, on the smaller dataset, the runtime difference between the two methods is negligible.

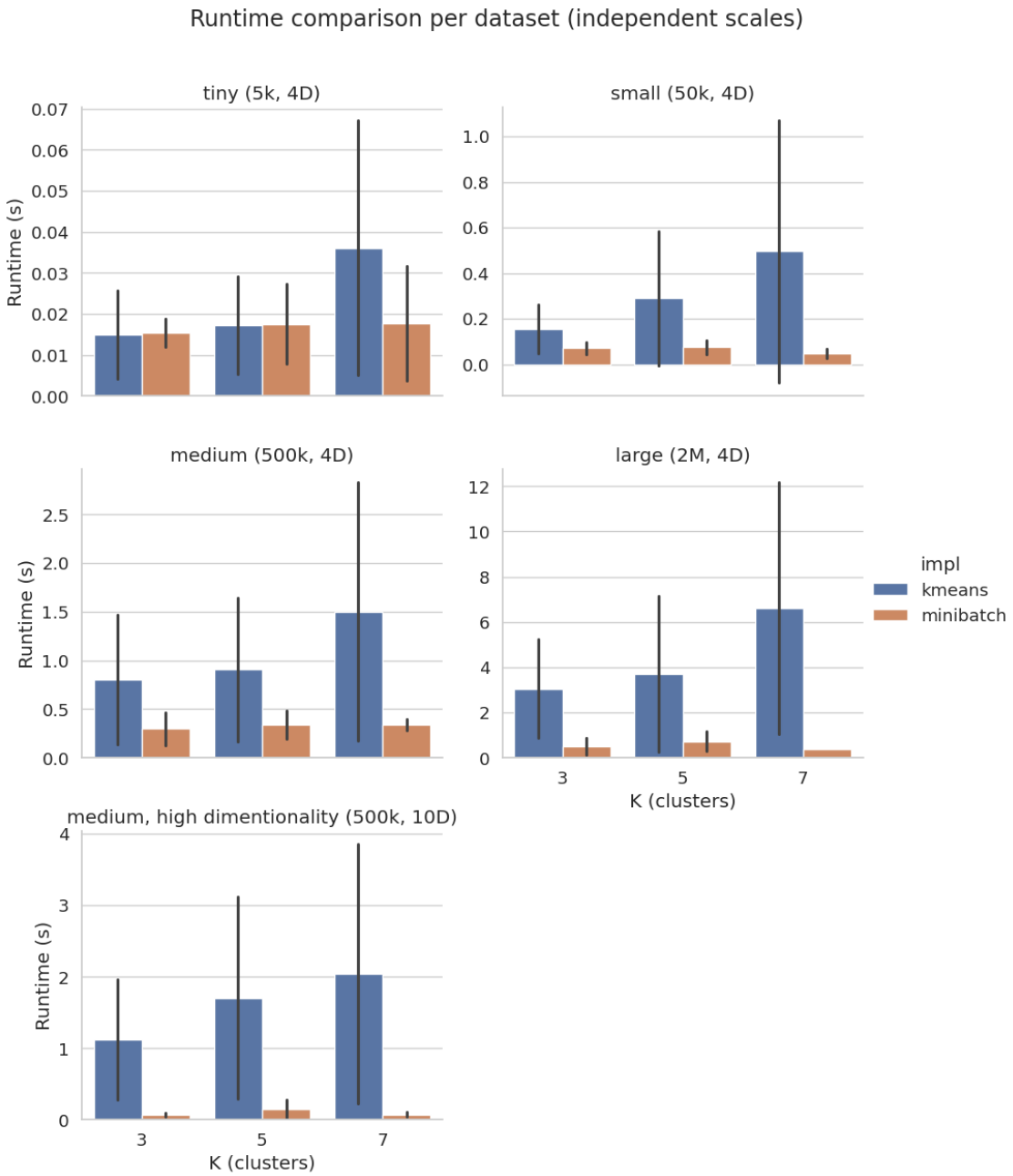


Figure 4: Runtime vs. number of clusters (K) for K-Means and MiniBatch K-Means.

Clustering quality. Across experiments, inertia decreased with higher k, as expected. MiniBatch K-Means achieved similar inertia values as full K-Means while running significantly faster, demonstrating that the approximation does not sacrifice clustering quality.

Inertia (elbow) analysis. Figure 5 reports inertia as a function of K for both full K-Means and MiniBatch K-Means. As expected, inertia decreases monotonically with K and exhibits an very slight"elbow" near the ground-truth cluster count. MiniBatch closely tracks full K-Means while remaining substantially faster, indicating that the approximation preserves clustering quality for practical choices of K.

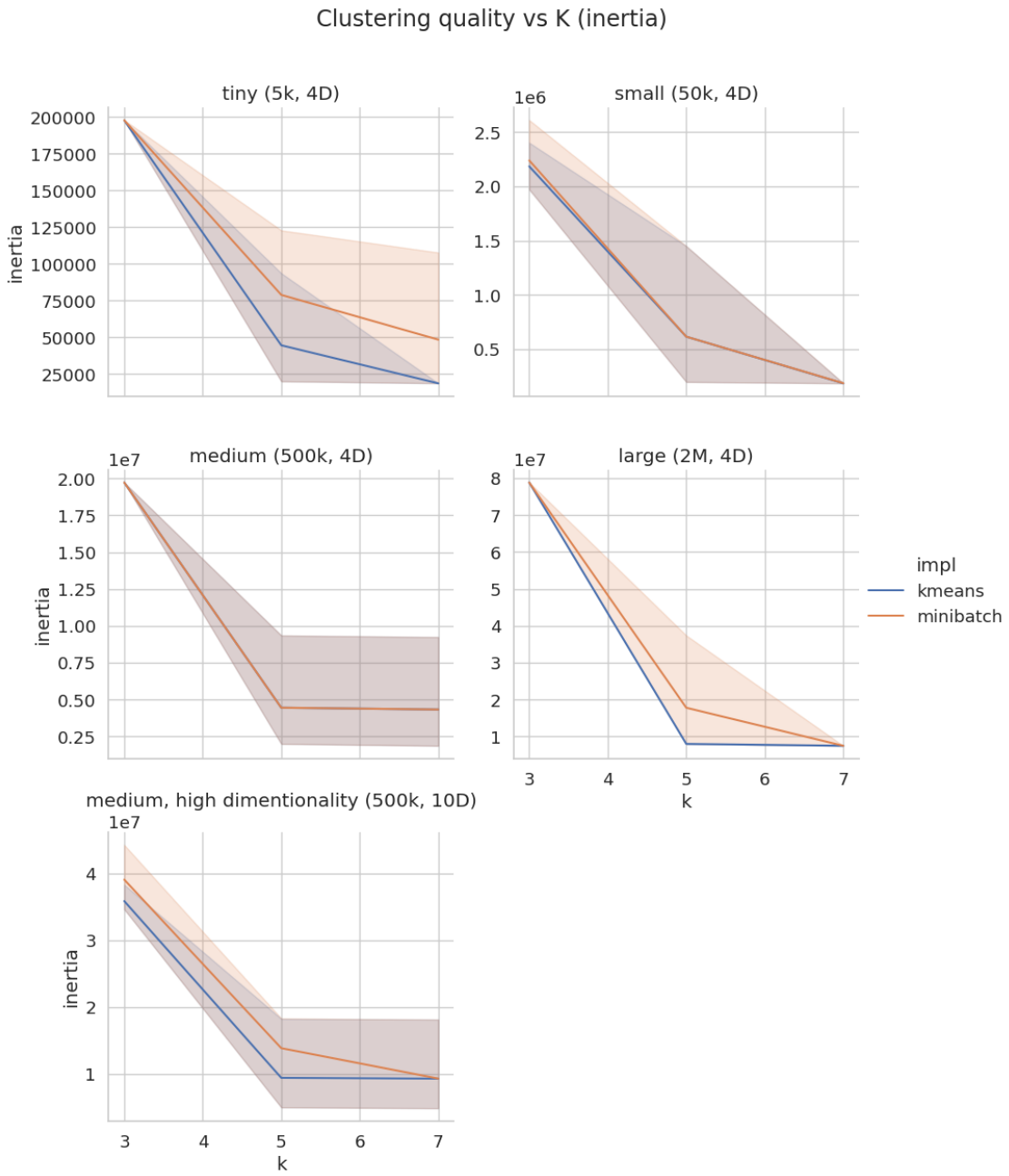


Figure 5: Inertia vs. number of clusters (K) for K-Means and MiniBatch K-Means (lower is better).

3.4 Discussion

These results confirm several known trade-offs:

- Increasing dataset size or dimensionality increases runtime almost linearly for full K-Means, while MiniBatch mitigates this growth effectively.
- Initialization strongly affects runtime and stability: `k-means++` is robust but slower, while naive strategies can degrade quality.

- Scaling to millions of points highlights the practical advantages of MiniBatch K-Means for big data scenarios.

The full notebook, including code, CSV result tables, and all figures, is provided in the submission files.

4 References

- Hadoop MapReduce Tutorial (1.x):
  https://hadoop.apache.org/docs/r1.2.1/mapred_tutorial.html
- HDFS Commands (3.3.6):
  https://hadoop.apache.org/docs/r3.3.6/hadoop-project-dist/hadoop-hdfs/HDFSCommands.html
- Project Gutenberg Offline Catalogs:
  https://www.gutenberg.org/ebooks/offline_catalogs.html
- Scikit-learn KMeans Documentation:
  https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html
- Scikit-learn MiniBatchKMeans Documentation:
  https://scikit-learn.org/stable/modules/generated/sklearn.cluster.MiniBatchKMeans.html
- Scikit-learn make_blobs (dataset generator):
  https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_blobs.html
- Matplotlib Pyplot (plotting library):
  https://matplotlib.org/stable/api/_as_gen/matplotlib.pyplot.html
- Seaborn API Reference (statistical visualization):
  https://seaborn.pydata.org/api.html